江西農業大學

JIANGXI AGRICULTURAL UNIVERSITY

本科毕业论文

题目： 基于微服务架构的高校智能选课系统设计与实现

学 院： 计算机与信息工程学院

姓 名： 彭益举

学 号： 6020222035

专 业： 软件工程

班 级： 软件工程 2022-1 班

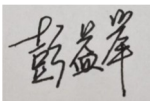
指导教师： 李佳航

答辩日期： 2026 年 04 月 19 日

江西农业大学

本科毕业论文原创性声明

本人郑重声明：所呈交的本科毕业论文是本人在导师的指导下独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本设计不包含任何其他个人或集体已经发表或撰写过的研究成果。对设计所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明或致谢。

作者签名： 

日　期：2026/3/11

目 录

摘 要.....I
Abstract.....II
第 1 章 绪 论.....1
1.1 课题研究背景及意义.....1
1.2 国内外研究现状.....1
1.3 论文的主要研究内容.....2
1.4 论文的组织结构.....3
1.5 本章小结.....4
第 2 章 系统开发相关技术分析.....5
2.1 相关技术.....5
2.1.1 Spring Boot 与 Spring Cloud Alibaba 微服务架构.....5
2.1.2 Nacos 服务注册与配置中心.....5
2.1.3 Redis 缓存与 Redisson 分布式锁.....5
2.1.4 RabbitMQ 消息队列与异步处理.....5
2.1.5 Sa-Token 统一鉴权框架.....6
2.1.6 MyBatis-Plus 数据访问框架.....6
2.1.7 React 与 Ant Design Pro 前端框架.....6
2.2 开发环境.....6
2.2.1 开发工具.....6
2.2.2 运行环境.....7
2.3 本章小结.....7
第 3 章 可行性与需求分析.....8
3.1 可行性分析.....8
3.1.1 技术可行性.....8
3.1.2 用户操作体验可行性.....8
3.1.3 开发成本可行性.....8
3.2 功能需求分析.....8
3.2.1 功能描述.....8
3.2.2 管理员用例图.....9
3.2.3 用户综合用例图.....10
3.3 本章小结.....11

第 4 章 系统概要设计.....12
4.1 系统架构设计.....12
4.2 总体结构设计.....12
4.2.1 用户服务层.....12
4.2.2 核心业务层.....12
4.2.3 数据支撑层.....13
4.3 功能模块设计.....13
4.3.1 智能课程推荐模块.....13
4.3.2 选课模块.....13
4.3.3 课程冲突检测模块.....14
4.3.4 系统管理模块.....14
4.3.5 消息通知与网关安全模块.....14
4.4 数据库设计.....15
4.4.1 概念设计.....15
4.4.2 逻辑设计.....15
4.5 本章小结.....18
第 5 章 系统详细设计与实现.....19
5.1 智能课程推荐模块.....19
5.1.1 推荐算法设计与实现.....19
5.1.2 推荐课程页面.....19
5.2 选课模块的实现.....20
5.2.1 高并发选课核心实现.....20
5.2.2 选课操作页面.....21
5.2.3 选课结果通知页面.....22
5.2.4 统一登录与网关转发流程.....23
5.3 课程冲突检测模块.....24
5.3.1 冲突检测算法实现.....24
5.4 系统管理模块的实现.....25
5.4.1 学生管理页面.....25
5.4.2 课程管理页面.....26
5.4.3 教师管理页面.....27
5.5 消息通知模块的实现.....28
5.6 系统安全模块的实现.....29

5.6.1 Sa-Token 登录与 Token 管理.....29
5.6.2 网关层鉴权过滤器.....29
5.7 本章小结.....30
第 6 章 系统测试.....32
6.1 测试设计.....32
6.2 主要功能测试.....32
6.2.1 系统安全功能测试.....32
6.2.2 智能推荐功能测试.....32
6.2.3 选课模块功能测试.....33
6.2.4 冲突检测功能测试.....33
6.2.5 系统管理模块功能测试.....34
6.2.6 消息通知模块功能测试.....34
6.3 性能压力测试.....34
6.4 本章小结.....35
第 7 章 总结与展望.....37
7.1 总结.....37
7.2 展望.....37
参考文献.....39
致谢.....40

摘 要

随着高等教育规模持续扩大，高校选课系统在学期初选课高峰期面临严峻的高并发压力。传统单体架构难以应对大规模并发访问、课程资源动态分配以及个性化推荐等多元需求，系统稳定性、数据一致性和用户体验均存在明显不足。大量高校在选课开放后的前几分钟内即出现服务器响应超时甚至宕机的情况，严重影响学生的正常选课流程，暴露出传统系统架构在突发高并发场景下的脆弱性。

本文设计并实现了一套基于微服务架构的高校智能选课系统，技术选型上以 Spring Boot 2.6.13 和 Spring Cloud Alibaba 为核心，Nacos 统一管理服务注册与配置，Spring Cloud Gateway 对外提供路由入口，Sa-Token 处理用户鉴权。选课并发控制方面，引入 Redisson 分布式锁保护库存扣减临界区，通过 RabbitMQ 将选课请求异步化从而平滑流量峰值，同时在选课服务内部记录 requestId 与处理状态，便于结果回查与异常排查。课程推荐模块根据学生专业字段为课程打分，专业对口课程给 90 分，通用选修课程给 50 分，降序取前 N 条作为推荐列表，并附上推荐理由文字；时间冲突检测则在原课程时间窗口基础上两端各延展 15 分钟，模拟课间转场所需时间，再与已选课程逐一比对。前端选用 React 结合 Ant Design Pro 构建管理界面，数据层使用 MyBatis-Plus 简化了大量重复的 CRUD 操作。

笔者使用 JMeter 对选课接口进行了阶梯并发压测，结果显示在 1000 并发用户下平均响应时间约 790ms，吞吐量约 1218 TPS，错误率维持在 2%以内，全程未出现一次课程超卖，说明 Redisson 分布式锁的并发保护策略是有效的，系统整体能够支撑高校大规模选课场景的实际需求。

关键词：微服务；智能推荐；高并发；选课系统；冲突检测

Abstract

With the continuous expansion of higher education, the university course selection system faces severe high-concurrency pressure at the beginning of each semester. Traditional monolithic architectures are inadequate to handle large-scale concurrent access, dynamic course resource allocation, and personalized recommendation requirements, leading to obvious deficiencies in system stability, data consistency, and user experience.

This paper designs and implements an intelligent course selection system based on microservice architecture, solving three core problems: system stability and data consistency under high concurrency, personalized intelligent course recommendation, and automatic time-conflict detection and warning. The system uses Spring Boot 2.6.13 with Spring Cloud Alibaba, Nacos for service registration and configuration, Spring Cloud Gateway for unified routing and authentication, and Sa-Token for unified authentication management. Redisson distributed locks protect stock deduction, RabbitMQ handles asynchronous request processing and notifications, and enrollment logs record request status for tracing. The recommendation module scores courses based on major match, assigning 90 points to matched courses and 50 points to general electives. The conflict detection module uses a time-window algorithm with 15-minute buffer thresholds. The frontend is built with React and Ant Design Pro, and the backend data layer uses MyBatis-Plus.

Testing results show that under 1000 concurrent users, the system maintains an average response time of 790ms, a throughput of 1218 TPS, and an error rate below 2%, with zero overselling incidents, meeting large-scale course selection requirements.

Key Words: Microservices; Intelligent Recommendation; High Concurrency; Course Selection; Conflict Detection

第 1 章 绪 论

1.1 课题研究背景及意义

在高等教育数字化转型的背景下，选课系统作为支撑教学管理的关键基础设施，其性能优化和稳定性保障面临着全新技术挑战。特别是在春季或秋季学期选课高峰期，系统需要同时应对大规模并发访问、课程资源动态分配以及个性化推荐等多重需求。传统的单体架构选课系统在高并发访问、数据安全和用户体验等方面存在诸多不足，尤其是在学期初选课高峰期，系统容易出现响应缓慢甚至崩溃的情况，严重影响学生的选课体验。据相关统计数据显示，全国高校在选课系统开放后的前 30 分钟内，系统请求量可瞬间增至日常访问量的数十倍，这对系统的并发承载能力提出了极高要求[1]。

传统选课系统通常采用集中式单体架构，将所有功能模块耦合在同一个部署单元中。这种架构在并发请求剧增时极易产生单点瓶颈，任何一个模块的性能下降都会影响整个系统的响应速度。更重要的是，传统系统普遍缺乏智能化的个性化推荐功能，学生需要在数百门可选课程中手动逐一筛选，效率低下，且容易遗漏与自身专业高度相关的优质课程。与此同时，课程时间冲突预警机制的缺失也导致学生在选课完成后才发现已选课程时间相互重叠，不得不重新进行选课调整，造成不必要的困扰。

为解决上述问题，本文引入 Spring Cloud Alibaba 微服务架构，通过服务解耦和独立部署实现功能模块的高内聚低耦合，显著提升系统的可扩展性与可维护性。同时，基于学生专业信息的智能课程推荐机制和时间窗口冲突检测算法为学生提供更加便捷、智能的选课体验，充分顺应了国家关于推进智慧校园建设的相关政策导向，为高校教务管理数字化转型提供有力技术支撑。

本研究的价值体现在以下三个层面：第一，技术创新层面，Redisson 分布式锁、RabbitMQ 消息队列、Redis 缓存的综合应用构建了高并发场景下的多重防护体系，显著提升了系统的抗压能力；第二，功能完善层面，智能推荐与冲突检测功能填补了传统系统的功能空白，显著提升了学生的选课体验与选课效率；第三，工程实践层面，本系统为高校教务信息化改造提供了可落地的微服务架构参考方案，具有较强的推广应用价值。

1.2 国内外研究现状

国内外关于高校选课系统的研究已取得一定进展，但在微服务架构、智能推荐与高并发处理的综合应用方面仍存在较大提升空间。

在国内研究方面，清华大学于 2018 年对原有选课系统进行了微服务化改造，采用 Spring Cloud 框架将选课、课程管理、用户认证等模块拆分为独立服务，通过

Nacos 进行服务注册与发现，系统并发承载能力提升 3 倍以上，选课高峰期系统稳定性得到显著改善[2]。复旦大学在智慧校园项目中将选课系统与教务管理信息系统深度整合，借助 Kubernetes 容器编排技术实现了服务弹性伸缩，并集成了基于协同过滤算法的课程推荐模块，学生选课满意度提升明显。中国人民大学则重点研究了基于 Redis 的分布式缓存方案，通过预加载热门课程库存信息将数据库查询压力降低了 70%以上，有效应对了选课高峰期的流量冲击。

在国外研究方面，斯坦福大学在线学习平台（Stanford Online）采用微服务架构管理多个学习服务模块，通过 Prometheus 与 Grafana 实现实时性能监控，并应用机器学习算法为学生提供个性化课程推荐，系统可用性达到 99.9%以上[3]。MIT 开放课程（OpenCourseWare）引入 Kubernetes 与 Spring Cloud，实现跨地域动态资源调度，保证了全球用户访问高峰期的服务稳定性。新加坡国立大学的选课系统研究聚焦于基于内容的过滤推荐算法，通过分析课程标签与学生历史选课记录的匹配度提升推荐准确性，推荐采纳率提升了约 35%[4]。

通过梳理国内外研究现状可以看出，现有高校选课系统研究仍存在若干共性不足。高并发防超卖方面，不少方案仍依赖数据库层面的乐观锁或悲观锁，在超高并发峰值下容易出现明显的性能瓶颈；课程推荐方面，部分研究倾向于采用深度学习等复杂模型，但这类方案对数据规模和服务器资源要求较高，在多数高校现有条件下推广成本较大；冲突检测功能方面，已有研究多停留在分钟级时间点比较，较少考虑学生在不同教学楼之间转场所需的现实时间。基于上述分析，本文选择以 Redisson 分布式锁配合 RabbitMQ 削峰的方式应对并发问题，以轻量、可解释的规则实现课程推荐，并在冲突检测中引入 15 分钟缓冲阈值，以兼顾技术可行性与工程落地成本。

1.3 论文的主要研究内容

本文的研究内容可以从以下几个模块来描述，各模块相互配合，共同构成一个完整的选课服务闭环。

（1）智能课程推荐模块。该模块以学生专业字段为核心匹配维度，对候选课程进行规则化评分。本专业相关课程赋予 90 分，通用选修课程赋予 50 分，再按分值降序返回前若干条推荐结果。每条推荐数据均附带解释性文字，例如"该课程与你的专业相关"，以增强推荐结果的可理解性。具体实现中，推荐服务通过 OpenFeign 跨服务调用选课微服务的分页接口获取课程数据，默认返回 5 条，前端也可通过 limit 参数自定义数量。

（2）高并发选课防护模块。针对选课高峰期的高并发问题，系统将选课服务从主业务服务独立部署，通过 Redisson 分布式锁确保同一课程在同一时刻只有一个请求进入库存扣减临界区。引入 RabbitMQ 消息队列将选课请求异步化处理，在

峰值期间实现流量削峰填谷；同时在选课服务内部记录 requestId 和处理状态，用于结果回查与异常排查。

（3）课程时间冲突检测模块。系统实现了基于时间窗口扩展的冲突检测算法，以课程默认时长 2 小时为基准，在课程开始与结束时间各设置 15 分钟的转场缓冲阈值，形成扩展时间区间。通过获取学生已选课程列表，逐一与待选课程扩展区间进行交叉比对，并根据时间差生成差异化冲突原因描述，为学生提供清晰的冲突预警信息。

（4）系统管理模块。涵盖学生管理、教师管理、课程管理三个子模块，按管理员、教师、学生三类用户区分功能入口与操作边界，支持课程二级审批流程（教师创建→管理员审核→上线可选），确保课程信息的规范性与教学质量标准。

（5）消息通知模块。选课结果无论成功还是失败均通过 RabbitMQ 消息队列异步推送至站内消息中心，通知内容以结果提示和失败原因说明为主，确保学生及时获取选课结果与课程变动信息。

（6）网关与安全模块。基于 Spring Cloud Gateway 实现统一路由转发与请求过滤，Sa-Token 框架负责登录态校验与会话管理，Nginx 通过 SSL/TLS 加密保障数据传输安全。

1.4 论文的组织结构

第一章绪论，从教育信息化建设现实需求出发，阐明本研究所要解决的核心问题，通过国内外文献调研指出现有研究的局限性，提出本文的研究方向，概述各章节主要内容。

第二章系统开发相关技术分析，深入介绍与系统开发相关的核心技术，包括 Spring Cloud Alibaba 微服务框架、Nacos 服务治理、Redis 缓存与 Redisson 分布式锁、RabbitMQ 消息队列、Sa-Token 鉴权、MyBatis-Plus 数据访问以及 React 前端技术栈，并详细说明系统开发环境配置。

第三章可行性与需求分析，从技术可行性、用户操作可行性和开发成本可行性三个维度论证系统建设的可行性，采用用例驱动方法梳理三类用户的功能需求，形成完整的需求规格说明。

第四章系统概要设计，阐述系统的整体四层架构设计，详细规划各功能模块的设计方案，完成数据库概念设计（E-R 图）和逻辑设计（七张核心表结构定义），为系统实现提供完整蓝图。

第五章系统详细设计与实现，以模块为单元通过核心代码解析与界面截图相结合的方式，系统性呈现六大模块的实现细节，验证各模块技术方案的可行性与功能完整性。

第六章系统测试，设计覆盖六大功能模块的测试用例，通过功能测试验证业

务逻辑正确性，借助 JMeter 阶梯式性能压力测试评估系统高并发处理能力。

第七章总结与展望，总结微服务架构在智能选课系统中的实践效果，并就推荐算法深化、弹性伸缩增强、移动端开发等方面提出未来演进方向。

1.5 本章小结

本章系统阐述了选课系统智能化改造的研究背景与重要意义，通过国内外文献调研梳理了现有研究成果的核心不足，提出了本论文在微服务架构下的六大核心模块设计思路，并清晰勾勒了全文的章节结构与论述脉络，为后续各章节的深入展开奠定了基础。

第 2 章 系统开发相关技术分析

2.1 相关技术

2.1.1 Spring Boot 与 Spring Cloud Alibaba 微服务架构

Spring Boot 由 Pivotal 团队提出，其核心理念是"约定优于配置"。借助自动装配机制，传统开发中大量依赖 XML 的配置工作得到显著简化。本系统基于 Spring Boot 2.6.13 版本开发，借助@SpringBootApplication 注解完成配置类、自动配置与包扫描的统一组织，并通过内嵌 Tomcat 实现快速启动与部署[5]。得益于丰富的 starter 体系，项目能够以较低的配置成本完成核心功能集成。

Spring Cloud Alibaba 是阿里巴巴基于 Spring Cloud 规范构建的扩展实现，能够将 Nacos、Sentinel 等国内常用中间件高效整合进微服务体系。本系统采用 Spring Cloud Alibaba 搭建微服务治理框架，具体包括通过 Nacos 完成服务发现、通过 OpenFeign 实现跨服务调用、通过 Nacos Config 集中管理公共配置等。相较于传统的 Eureka 与 Config Server 组合，该方案在当前项目场景下具有更高的集成效率与配置便利性。

2.1.2 Nacos 服务注册与配置中心

Nacos 全称 Dynamic Naming and Configuration Service，是阿里巴巴开源的一款服务发现与配置中心，在国内微服务项目中应用较为广泛[6]。在本系统中，Nacos 主要承担两项职责：其一，作为服务注册中心，各微服务启动后将自身地址与端口信息注册至 Nacos，Spring Cloud Gateway 据此完成动态路由，无需手工维护服务地址；其二，作为配置中心，数据库连接、Redis 地址等公共配置统一存放于 Nacos，各服务启动时按需拉取，结合@RefreshScope 注解后可支持配置热更新。

2.1.3 Redis 缓存与 Redisson 分布式锁

Redis（Remote Dictionary Server）是一款基于内存的高性能键值存储系统，支持 String、Hash、List、Set、Sorted Set 等丰富数据结构，以亚毫秒级响应延迟著称。在本系统中，Redis 主要承担两项职责：一是作为 Sa-Token 与相关会话组件的后端存储，实现多服务实例间的登录态共享；二是作为 Redisson 分布式锁的底层协调介质，为选课临界区并发控制提供支撑[7]。

Redisson 是构建在 Redis 之上的 Java 分布式对象与服务框架，提供了 RLock（分布式锁）、RSemaphore 等高级并发原语。本系统在选课核心流程中使用 RLock 实现分布式锁，锁定键格式为"curriculum:_select_{curriculumId}"，通过 tryLock(500ms 等待, 5000ms 租约)机制确保同一课程在任意时刻只有一个选课请求进入临界区执行库存扣减，从而降低并发超卖风险。

2.1.4 RabbitMQ 消息队列与异步处理

RabbitMQ 是基于 AMQP 协议的开源消息代理，以可靠投递、灵活的交换机路由和完善的管理界面而广泛应用于分布式系统的异步解耦场景[8]。本系统引入 RabbitMQ 实现双重目标：其一，流量削峰，主业务服务将选课请求封装为 EnrollmentDTO 消息发送至队列后立即返回"处理中"响应，选课微服务从队列中按序消费并执行实际的选课逻辑，将峰值并发压力从数据库层面平移至消息队列层面；其二，异步通知，选课结果（成功/失败及原因）通过通知队列异步发送至站内消息中心，实现了选课处理与通知推送的彻底解耦，确保通知的可靠投递。

2.1.5 Sa-Token 统一鉴权框架

Sa-Token 是一个轻量级 Java 权限认证框架，以简洁的 API 设计和开箱即用的功能特性著称，可通过数行代码完成登录认证、权限校验等操作[9]。本系统通过 Sa-Token 实现统一鉴权：用户登录后由 StpUtil.login(userId)生成会话 Token 并结合 Redis 进行存储；网关层通过自定义 SaReactorFilter 拦截请求，对登录、注册等白名单接口放行，其余接口调用 StpUtil.checkLogin()验证 Token 有效性。当前配置中 Token 有效期为 30 天，active-timeout 为-1，整体策略更偏向于保证开发和演示阶段的登录连续性。

2.1.6 MyBatis-Plus 数据访问框架

MyBatis-Plus 是在 MyBatis 基础上进行功能增强的 ORM 框架，提供了通用 Mapper（BaseMapper<T>）、通用 Service（IService<T>）、条件构造器（LambdaQueryWrapper）等特性，将单表 CRUD 操作的样板代码量减少 80%以上。本系统全面采用 MyBatis-Plus 进行数据访问，通过@TableLogic 注解结合 is_delete 字段实现逻辑删除，@TableField(fill=FieldFill.INSERT)注解自动填充 create_time 与 update_time 字段，规范化数据操作行为，大幅提升开发效率。

2.1.7 React 与 Ant Design Pro 前端框架

React 是由 Meta 开发的声明式、组件化 JavaScript 视图库，通过虚拟 DOM 差分算法实现高效的 UI 局部更新，结合 Hooks 特性实现函数式组件状态管理，是当前最主流的前端框架之一[10]。Ant Design Pro 是基于 Ant Design 企业级设计体系的前端应用框架，提供了布局方案、权限管理等开箱即用功能，内置 ProTable（高级表格）、ProForm（高级表单）等业务组件，可大幅缩短管理后台的开发周期。本系统前端采用 React + TypeScript + Ant Design Pro 技术栈，通过 UmiJS 框架管理路由，使用 Axios 封装 HTTP 请求，系统界面自适应 PC 与移动端多种分辨率。

2.2 开发环境

2.2.1 开发工具

集成开发环境：后端开发采用 IntelliJ IDEA 2023 Ultimate 版，通过深度集成的 Spring Boot 插件、Nacos 工具、Docker 支持，实现从微服务编码到容器化构建的全

流程开发。前端开发采用 VS Code，配合 ESLint、TypeScript 语言服务和 Ant Design Pro 代码片段，提升界面开发效率与代码质量。版本管理采用 Git，通过 Gitee 托管代码仓库，执行特性分支开发模式，commit message 遵循约定式规范。数据库管理：Navicat Premium 15 用于 MySQL 管理；Another Redis Desktop Manager 用于 Redis 监控；Postman 用于 API 接口调试。

2.2.2 运行环境

后端按 pom 配置使用 Java 8，构建工具是 Maven 3.9。中间件方面，MySQL 8.0 存储业务数据，Redis 用于会话与分布式锁支撑，RabbitMQ 负责消息队列并开启了消息持久化配置，Nacos 2.x 作为注册与配置中心。前端基于 Node.js 运行环境，用 npm 管理依赖，锁定 package-lock.json 来保证开发和生产环境的一致性。UmiJS 内置 Webpack 做打包，支持按路由懒加载。安全方面，部署阶段可由 Nginx 承担 HTTPS 终止，日志通过 SLF4J + Logback 输出，方便排查问题。

2.3 本章小结

本章系统梳理了选课系统开发所依赖的核心技术体系，从微服务框架（Spring Boot + Spring Cloud Alibaba）、服务治理（Nacos）、缓存与分布式锁（Redis + Redisson）、消息队列（RabbitMQ）、安全认证（Sa-Token）、数据访问（MyBatis-Plus）到前端技术栈（React + Ant Design Pro）进行了全面深入的介绍。这些技术的有机组合为本系统高可用、高并发、易扩展的目标提供了坚实的技术基础，为后续系统设计与实现奠定了理论依据。

第 3 章 可行性与需求分析

3.1 可行性分析

3.1.1 技术可行性

本系统所采用的技术组件均已在大量生产环境中得到充分验证，技术成熟度高，社区活跃，文档完善，具备充分的技术可行性。在高并发防护方面，Redisson 分布式锁已被广泛用于秒杀、抢购等高并发场景，是解决库存超卖问题的成熟方案；RabbitMQ 用于将瞬时请求压力从同步链路转移到异步处理链路；Spring Cloud Gateway 负责统一接入和登录态校验，三者组合能够满足本系统的工程实现需求。

在推荐算法的可行性方面，本文有意避开机器学习方向。原因在于高校服务器资源相对有限，训练数据规模也较为有限，若直接引入复杂模型，收益与成本并不匹配。最终采用基于专业字段的规则匹配与评分方法，算法复杂度为 O(n log n)，计算过程无需额外模型服务支撑，可直接在 Java 服务中完成。冲突检测方面，区间交叉判断属于基础算法，一个学生的已选课程数量通常有限，逐门遍历即可满足业务需要。总体来看，这两类功能对现有软硬件环境要求较低，具备较强的技术可行性。

3.1.2 用户操作体验可行性

系统采用 Ant Design Pro 的响应式栅格布局，在 PC、平板、手机三类终端设备上均能提供一致的操作体验，核心功能适配率达 100%。学生端通过"一键选课"设计将传统多步骤流程简化为"筛选课程→点击选课→弹窗确认"三步操作，并在确认弹窗中集成冲突检测结果展示，帮助学生提交前充分了解潜在风险。个性化推荐列表以卡片形式呈现，清晰标注推荐理由，有效降低学生的选课决策成本。管理员端关键操作控制在 3 步以内，系统响应时间控制在 1 秒内，符合教育部相关信息化系统可用性标准。

3.1.3 开发成本可行性

本系统由笔者独立开发完成，所采用技术栈均为开源方案，软件授权成本较低。开发工具方面，后端使用学校提供教育授权的 IntelliJ IDEA，前端使用 VS Code。部署资源方面，系统运行于一台 2 核 4G 基础云服务器，中间件通过 Docker Compose 统一编排。综合服务器、域名及其他支出，全年总成本控制在 300 元以内，符合毕业设计项目对开发成本的预期要求。

3.2 功能需求分析

3.2.1 功能描述

系统面向三类用户，不同角色的功能需求各有侧重。学生是系统的高频使用

者，其核心诉求包括快速定位目标课程、在提交前获知潜在时间冲突以及在选课完成后及时接收结果通知。具体功能包括：按课程名称、教师、授课时间、专业方向等多个条件筛选课程；查看系统基于专业生成的推荐列表及推荐理由；在确认选课前弹出时间冲突检测结果；查看个人课程表；接收选课成功或失败的站内通知；以及执行退课操作。系统设计还需满足较高并发访问场景下的稳定运行要求。

教师用户核心需求：课程信息发布与维护（填写课程名称、学分、授课时间、地点、适用专业等属性后提交审核）；课程信息修改（修改后需重新触发审核）；选课学生名单查看与数据导出；接收课程审核通过/驳回通知。管理员用户核心需求：学生与教师信息全生命周期管理（增删改查）；课程信息审核（通过/驳回，可附驳回原因）与强制扩容；未读消息管理。

3.2.2 管理员用例图

管理员在系统里主要扮演运维和审核的角色，权限最高。他需要能管理教师和学生账户，处理课程审核请求（通过或者驳回，驳回时要给出理由），必要时对名额不足的课程扩容，并维护系统基础数据。管理员用例图如图 3.1 所示。

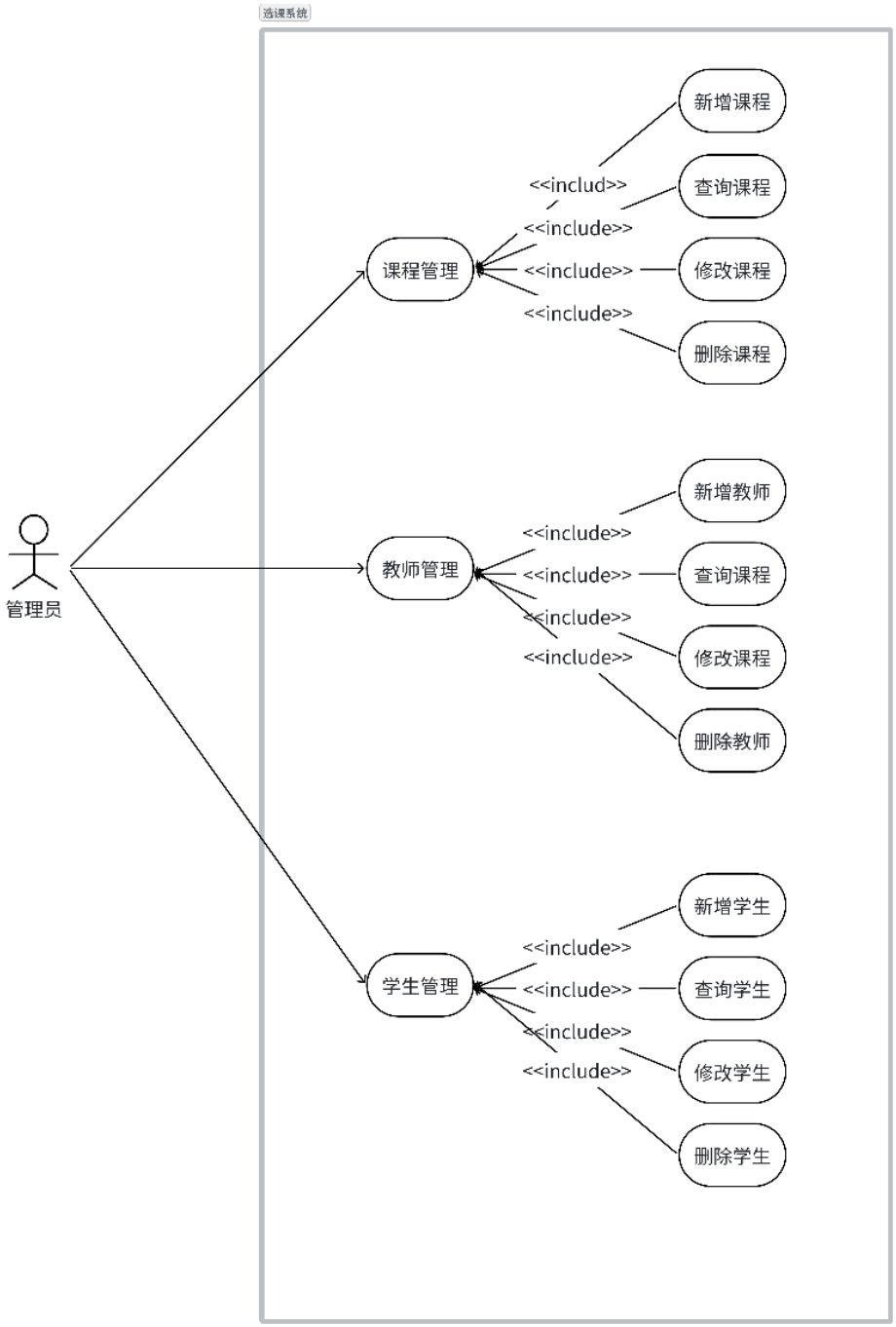


图 3.1 管理员用例图

3.2.3 用户综合用例图

学生角色完成选课全流程：通过多条件筛选定位目标课程，系统在提交选课前自动执行时间冲突检测并以弹窗展示检测结果，学生确认无冲突后提交，系统异步处理后通过消息中心推送选课结果；课后可通过课表页面管理已选课程。教师角色聚焦课程管理，提交审核申请，待管理员审核通过后课程上线供学生选择。用户综合用例图如图 3.2 所示。

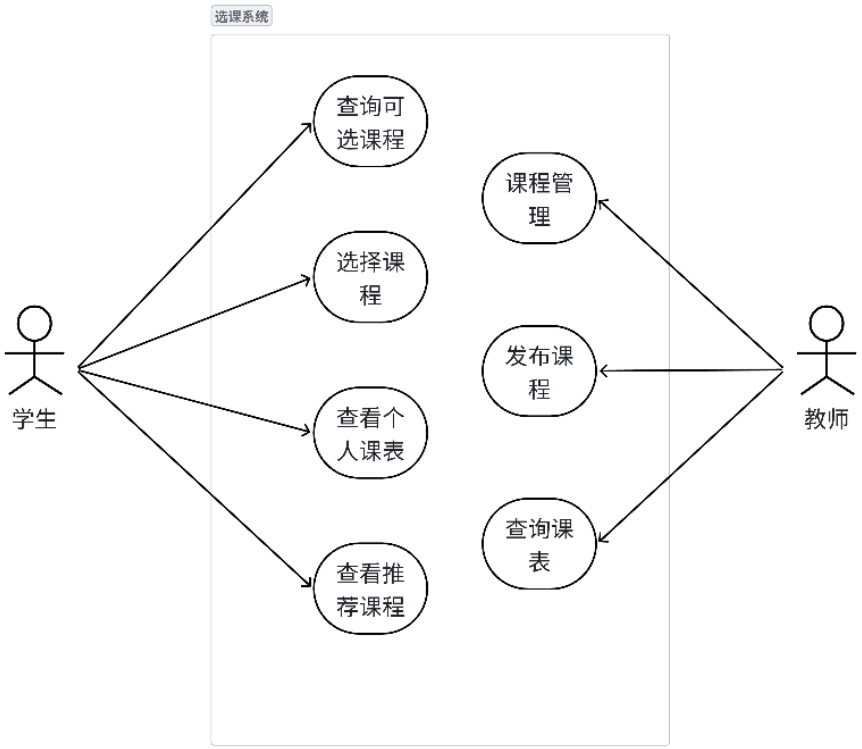


图 3.2 用户综合用例图

3.3 本章小结

本章从技术可行性、用户体验和开发成本三个角度论证了系统建设的合理性，所有选用的技术组件都有成熟的社区支撑，个人开发成本也控制在合理范围内。需求方面，以用例图的形式梳理了三类用户的业务诉求，核心功能边界已经明确，为后续的设计和开发工作提供了依据。

第 4 章 系统概要设计

4.1 系统架构设计

系统整体采用四层架构设计，各层职责边界明确。

最外层是访问层，三类用户通过浏览器发起 HTTPS 请求，前端打包后的静态文件由 Nginx 托管，所有通信强制走 TLS 加密，防止数据在传输过程中被截取。

接入层有两个角色：Nginx 在前负责静态资源分发和隐藏后端真实 IP；Spring Cloud Gateway 在后充当微服务网关，通过 Nacos 服务发现动态路由到各微服务实例。所有请求在进入业务服务前先经过 Sa-Token 过滤器做登录态校验，并统一处理跨域与路由转发。

应用层：主业务服务（main-service，端口 8101）处理低频业务，包括用户管理、课程管理、智能推荐、冲突检测等；选课微服务（select-service，端口 8081）专注处理高频高并发的选课业务，通过 Redisson 分布式锁与 RabbitMQ 保障数据一致性。两个服务均向 Nacos 注册，通过 OpenFeign 实现跨服务远程调用。

数据存储层：MySQL 8.0 提供可靠的关系型业务数据持久化存储；Redis 6.0 提供高速内存缓存与分布式会话存储；RabbitMQ 承担异步消息传递与流量削峰职责，三者协同形成高性能、高可靠的数据支撑体系。

4.2 总体结构设计

4.2.1 用户服务层

三类用户的功能入口与界面设计具有明显差异。学生端提供课程查询、推荐课程、选课和课程表四个主要功能模块，选课结果通过消息中心通知，并在提交前展示冲突检测结果。教师端主要承担课程发布与维护工作，提交后进入审核流程，同时可查看选课名单。管理员端权限最高，负责账户管理、课程审核及基础运维等管理工作。

4.2.2 核心业务层

（1）智能流量调控体系：Spring Cloud Gateway 统一入口，RabbitMQ 消息队列承担请求削峰，Redisson 分布式锁负责选课临界区并发控制。三者组合将高峰期的同步压力转移到更可控的异步处理链路中。

（2）智能选课引擎：包含三个核心子功能：个性化推荐（专业匹配评分，高相关性课程优先）、冲突预检（提交前自动遍历已选课程进行时间区间交叉比对）、库存管理（课程库存字段配合分布式锁控制名额扣减）。

（3）综合安全保障体系：Sa-Token 与 Spring Cloud Gateway 负责统一登录校验，用户角色字段用于区分管理员、教师、学生三类账户，Nginx 通过 SSL/TLS 加密防止数据传输过程中被窃听或篡改。

4.2.3 数据支撑层

数据存储体系：业务数据库存储课程、选课记录、用户档案等核心数据；选课日志表记录每次异步处理对应的 requestId 和状态，用于结果回查与问题溯源。数据一致性机制：选课成功后即时更新课程库存与选课记录；消息通知落库后同步更新未读消息计数。数据分析服务：选课热点统计识别高频访问课程，为资源分配提供数据依据；选课成功率统计用于系统性能优化参考。

4.3 功能模块设计

4.3.1 智能课程推荐模块

推荐模块以学生专业为匹配维度，通过 OpenFeign 调用选课微服务获取课程分页数据，对专业对口课程赋值 90 分，对通用选修课程赋值 50 分，再按得分排序后返回前 N 条结果。若专业匹配课程数量不足设定的推荐数，则以通用课程补足。每条推荐结果均附带推荐理由，例如"该课程与你的专业相关"或"通用选修课程"。系统默认返回 5 条推荐结果，前端可通过 limit 参数进行调整。

4.3.2 选课模块

选课模块需要重点解决高并发场景下的数据一致性问题。其处理流程如下：主业务服务先将选课请求投递至 RabbitMQ 队列并立即返回"处理中"提示；异步消费者为本次处理生成 requestId 后调用选课微服务；选课微服务内部先查询 enrollment_log 中的同 requestId 记录，再尝试获取 Redisson 分布式锁，等待上限 500ms，持锁最长 5000ms；加锁成功后继续校验库存是否大于 0，以及该学生是否已经选过该课程；两项校验均通过后写入选课记录并扣减库存；若抢锁失败则返回"系统繁忙，请稍后重试"。整个流程完成后，选课结果通过 RabbitMQ 异步发送通知。选课模块时序图如图 4.1 所示。

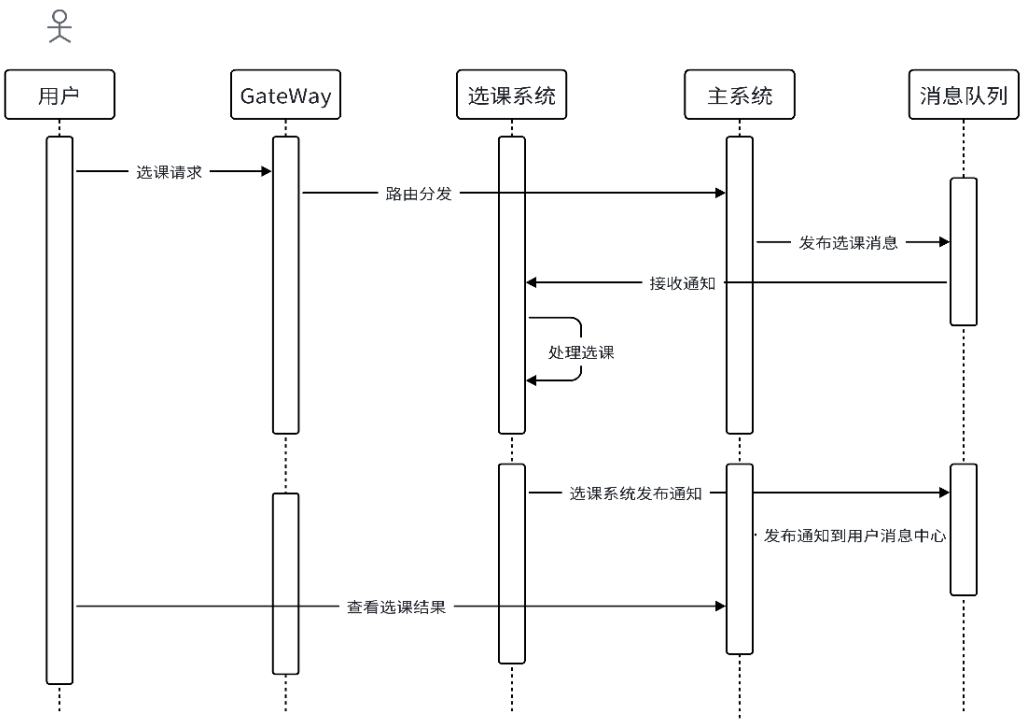


图 4.1 选课模块时序图

4.3.3 课程冲突检测模块

冲突检测的核心是时间窗口扩展算法。考虑到学生在不同教学楼之间转场通常需要 10 至 15 分钟，若仅采用精确时间点匹配，可能遗漏"形式上不冲突、实际无法及时到达"的情况。因此，系统将目标课程的开始时间提前 15 分钟、结束时间延后 15 分钟，形成扩展区间，再与已选课程逐一进行交叉比对。交叉判断覆盖三种情形：已选课程的开始时间落在扩展区间内、结束时间落在扩展区间内，或完全包含扩展区间。检测到冲突后返回冲突课程列表和原因说明，用于对主要时间重叠场景进行预警。

4.3.4 系统管理模块

系统管理模块按角色划分为三层权限：管理员具备完整管理权限；教师仅可维护本人创建的课程；学生仅保留查询类权限。课程上线遵循明确的审批流程：教师创建课程并提交后，课程处于"待审核"状态（isCheck=0），此时不对学生展示；管理员审核通过后状态更新为 1，课程方可进入学生查询列表；若审核驳回则状态更新为 2，并可填写驳回原因，教师根据意见修改后可再次提交。

4.3.5 消息通知与网关安全模块

消息通知模块采用 RabbitMQ 实现通知的异步可靠发送。选课处理完成后，系统向通知队列发送消息，消费者接收后写入 message 表，同时对用户的未读消息数字段执行加一操作。前端采用定时轮询方式每隔 30 秒查询一次未读数，若存在新消息，则在导航栏显示红点提示。网关安全方面，Spring Cloud Gateway 负责统一

入口鉴权与跨域处理，Sa-Token 负责登录态校验，Nginx 配合 SSL 证书保障传输安全。

4.4 数据库设计

4.4.1 概念设计

系统涉及的核心数据实体有七类：用户、学生、教师、课程（Subject）、课程安排（Curriculum）、选课记录（Enrollment）、选课日志（EnrollmentLog）。课程安排和学生之间是多对多关系，通过选课记录表来桥接。选课日志单独建表，记录每次选课操作的请求 ID 和处理状态，用于结果回查与异常排查。系统 E-R 图如图 4.2 所示。

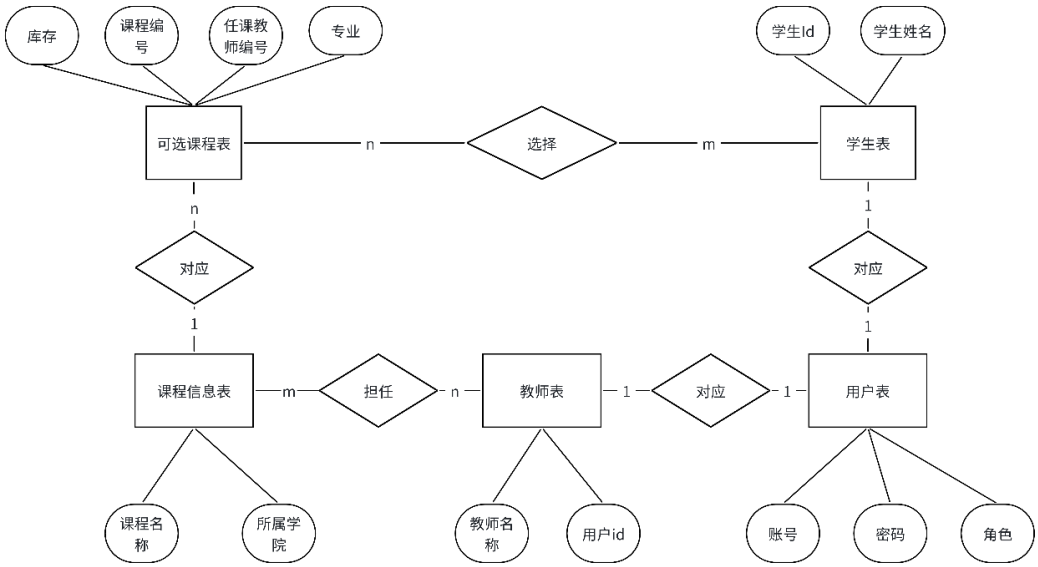


图 4.2 系统 E-R 图

4.4.2 逻辑设计

（1）用户表（user）—— 存储所有用户的账户信息与基础属性，通过 userRole 字段区分管理员、教师、学生三类角色。该表同时保存个性签名、头像、性别以及未读消息计数等通用信息。

表 4.1 用户表（user）

字段名	字段描述	数据类型	长度	是否主键
id	主键	bigint	-	是
userName	用户昵称	varchar	256	否
userAccount	登录账号	varchar	256	否
userPassword	登录密码	varchar	512	否
userDescription	个性签名	varchar	255	否
userAvatar	用户头像 URL	varchar	1024	否
gender	性别	tinyint	-	否
userRole	用户角色	varchar	256	否

createTime	创建时间	datetime	-	否
updateTime	更新时间	datetime	-	否
isDelete	逻辑删除	tinyint	-	否
unReadMessage	未读消息数	int	-	否

（2）学生表（student）——存储学生特有信息，通过 userId 与用户表关联。除学号、姓名外，还保存年级、学院、联系方式等信息，其中 major 字段直接参与推荐算法匹配。

表 4.2 学生表（student）

字段名	字段描述	数据类型	长度	是否主键
id	主键	int	-	是
studentId	学号	bigint	-	否
userId	关联用户 ID	bigint	-	否
studentName	学生姓名	varchar	50	否
gender	性别	varchar	10	否
age	年龄	int	-	否
grade	年级	varchar	20	否
major	专业	varchar	50	否
college	学院	varchar	50	否
phoneNumber	联系电话	varchar	20	否
email	邮箱	varchar	50	否
createTime	创建时间	datetime	-	否
updateTime	更新时间	datetime	-	否

（3）教师表（teacher）——存储教师基本信息。

表 4.3 教师表（teacher）

字段名	字段描述	数据类型	长度	是否主键
id	主键	int	-	是
teacherId	工号	bigint	-	否
userId	关联用户 ID	bigint	-	否
teacherName	教师姓名	varchar	50	否
level	职称	varchar	20	否

（4）课程表（subject）——存储课程基本信息，课程名称、开课学院和学分均在该表中维护。

表 4.4 课程表（subject）

字段名	字段描述	数据类型	长度	是否主键
id	主键	int	-	是
subjectName	课程名称	varchar	50	否

college	开课学院	varchar	50	否
credit	课程学分	double	-	否
createTime	创建时间	datetime	-	否
updateTime	更新时间	datetime	-	否

（5）课程安排表（curriculum）——存储具体课程安排。该表除授课时间、地点、适用专业外，还维护星期、节次、审核状态以及总名额和剩余名额，用于支撑冲突检测、审核流与库存控制。

表 4.5 课程安排表（curriculum）

字段名	字段描述	数据类型	长度	是否主键
id	主键	int	-	是
subjectId	课程 ID	int	-	否
teacherId	教师 ID	int	-	否
teachingTime	授课时间	datetime	-	否
dayOfWeek	星期几(1-7)	int	-	否
startTime	开始节次	int	-	否
endTime	结束节次	int	-	否
location	授课地点	varchar	50	否
remarks	课程备注	varchar	255	否
grade	适用年级	varchar	20	否
major	适用专业	varchar	50	否
isCheck	审核状态(0 待/1 过/2 驳)	tinyint	-	否
stock	剩余名额	int	-	否
isStock	是否限额	int	-	否
createTime	创建时间	datetime	-	否
updateTime	更新时间	datetime	-	否
isDelete	逻辑删除	tinyint	-	否
allStock	课程总名额	int	-	否

（6）选课记录表（enrollment）——记录学生与课程安排的多对多关联关系。

表 4.6 选课记录表（enrollment）

字段名	字段描述	数据类型	长度	是否主键
id	主键	bigint	-	是
studentId	学生 ID	bigint	-	否
curriculumId	课程安排 ID	bigint	-	否
createTime	创建时间	datetime	-	否
updateTime	更新时间	datetime	-	否

（7）选课日志表（enrollment_log）——记录所有选课操作历史，用于处理状态记录与问题追溯。

表 4.7 选课日志表（enrollment_log）

字段名	字段描述	数据类型	长度	是否主键
id	主键	bigint	-	是
requestId	请求处理 ID	bigint	-	否
studentId	学生 ID	bigint	-	否
status	操作状态(0 处理中/1 成功/2 失败)	tinyint	-	否
createTime	创建时间	datetime	-	否
updateTime	更新时间	datetime	-	否

4.5 本章小结

本章从四层系统架构、三大业务体系（流量调控/选课引擎/安全保障）到七张数据库表结构，完整呈现了系统的概要设计方案。四层架构设计通过关注点分离实现了安全防护、性能优化与弹性扩展的有机统一；功能模块设计充分考虑了高校选课业务的特殊需求；规范化的数据库表结构为系统功能实现提供了可靠的数据基础，为后续详细设计与编码实现提供了清晰蓝图。

第 5 章 系统详细设计与实现

5.1 智能课程推荐模块

5.1.1 推荐算法设计与实现

智能推荐的核心逻辑位于 CourseRecommendationService 类中。系统首先根据 studentId 查询学生专业信息，再通过 OpenFeign 调用选课微服务的分页接口获取按专业过滤的课程列表，并赋予 90 分；若专业匹配课程数量不足预设 limit，则补充通用课程并赋予 50 分；最后对所有候选课程按得分降序排列，截取前 limit 条返回。主服务不单独维护课程数据副本，课程信息均由选课微服务实时提供，以保证数据一致性。核心代码如下。

```
public List<CourseRecommendVO> recommendCoursesForStudent(
        Long studentId, Integer limit) {
    if (limit == null || limit <= 0) limit = 5;
    Student student = studentMapper.selectById(studentId);
    if (student == null) return Collections.emptyList();

    List<CourseRecommendVO> result = new ArrayList<>();
    // Step1: 按专业查询匹配课程(推荐分 90.0)
    if (student.getMajor() != null) {
        params.put("major", student.getMajor());
        PageParam<Curriculum> majorPage = enrollmentClient.page(params);
        for (Curriculum c : majorPage.getRecords()) {
            CourseRecommendVO vo = buildVO(c, student.getMajor());
            vo.setRecommendScore(90.0);
            result.add(vo);
        }
    }
    // Step2: 不足时以通用选修补充（推荐分 50.0）
    // Step3: 按推荐分降序，截取前 limit 条返回
    result.sort(Comparator.comparing(
        CourseRecommendVO::getRecommendScore).reversed());
    return result.subList(0, Math.min(result.size(), limit));
}
```

这段代码跨服务调用通过 Feign 完成，整个逻辑在 Java 端跑，不依赖外部模型服务。排序用的是 Comparator.comparing 结合 reversed()，复杂度 O(n log n)。高校场景下课程总量通常有限，这类基于规则的推荐方式能够在较低计算开销下给出稳定结果，也便于后续继续扩展更多推荐维度。

5.1.2 推荐课程页面

推荐页面采用卡片网格布局展示结果，每张卡片显示课程名称、授课时间、

授课地点、适用专业及推荐理由，右下角设置"立即选课"按钮以便直接进入选课流程。页面加载完成后自动请求推荐接口，在 loading 阶段展示 Ant Design 的 Spin 组件，数据返回后渲染卡片列表。推荐课程页面如图 5.1 所示。

图 5.1 推荐课程页面

5.2 选课模块的实现

5.2.1 高并发选课核心实现

选课的核心控制器位于 EnrollmentController 类中。为保证高并发场景下的处理稳定性，系统主要依赖三道防线：requestId 处理日志记录状态、Redisson 分布式锁防止并发超卖、库存校验作为兜底机制。完整核心代码如下。

```
@PostMapping("/select")
public String enrollStudentToCourse(Long studentId,
        Long curriculumId, Long requestId) {
    // 第一步：查询处理日志
    EnrollmentLog log = enrollmentLogService.getOne(
        new LambdaQueryWrapper<EnrollmentLog>()
            .eq(EnrollmentLog::getRequestId, requestId));
    if (log != null) {
        return log.getStatus()==1 ? "已经选课成功" : "系统繁忙,请稍后重试";
    }
    // 第二步：获取 Redisson 分布式锁
    RLock lock = redissonClient.getLock(
        "curriculum:_select_" + curriculumId);
    try {
        boolean locked = lock.tryLock(500, 5000, TimeUnit.MILLISECONDS);
        if (locked) {
```

```
                Curriculum c = curriculumService.getOne(...);
                if (c.getStock() <= 0) return "选课失败,该课程人数已满";
                if (enrollmentService.getOne(...) != null)
                    return "选课失败，你已经选过该课程";
                EnrollmentLog newLog = new EnrollmentLog();
                newLog.setRequestId(requestId);
                newLog.setStudentId(studentId);
                newLog.setStatus(0);
                enrollmentLogService.save(newLog);
                return enrollmentService.enrollStudentToCourse(
                    studentId, c, newLog);
            }
            return "系统繁忙，请稍后再试";
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return "服务器出了点小差，请稍后再试";
        } finally {
            if (lock.isHeldByCurrentThread()) lock.unlock();
        }
    }
```

tryLock(500, 5000, TimeUnit.MILLISECONDS)的参数含义：500ms 为最长等待获取锁的时间，超时则放弃并返回"系统繁忙"；5000ms 为锁的自动释放时间（租约），防止持锁服务崩溃后长期占锁。这里的 requestId 并不是前端直接传入的业务参数，而是异步消费者在调用选课微服务前生成的内部处理标识，并同步写入 enrollment_log 用于记录本次处理状态。主业务服务通过 rabbitTemplate.convertAndSend()将 EnrollmentDTO 发送至 RabbitMQ 队列实现异步处理，前端在收到"处理中"响应后展示等待提示，选课结果在 0 至 5 分钟内通过消息中心通知学生。

5.2.2 选课操作页面

进入选课页面后，学生通过顶部搜索栏输入课程名称或教师姓名进行筛选，课程列表以分页表格形式展示（每页 20 条）。点击目标课程右侧"选择课程"按钮，系统弹出确认对话框，显示课程完整信息及冲突检测结果。学生确认后点击"确认选课"，界面立即反馈"正在处理中"状态提示。选课页面如图 5.2 所示，选课操作确认弹窗如图 5.3 所示。

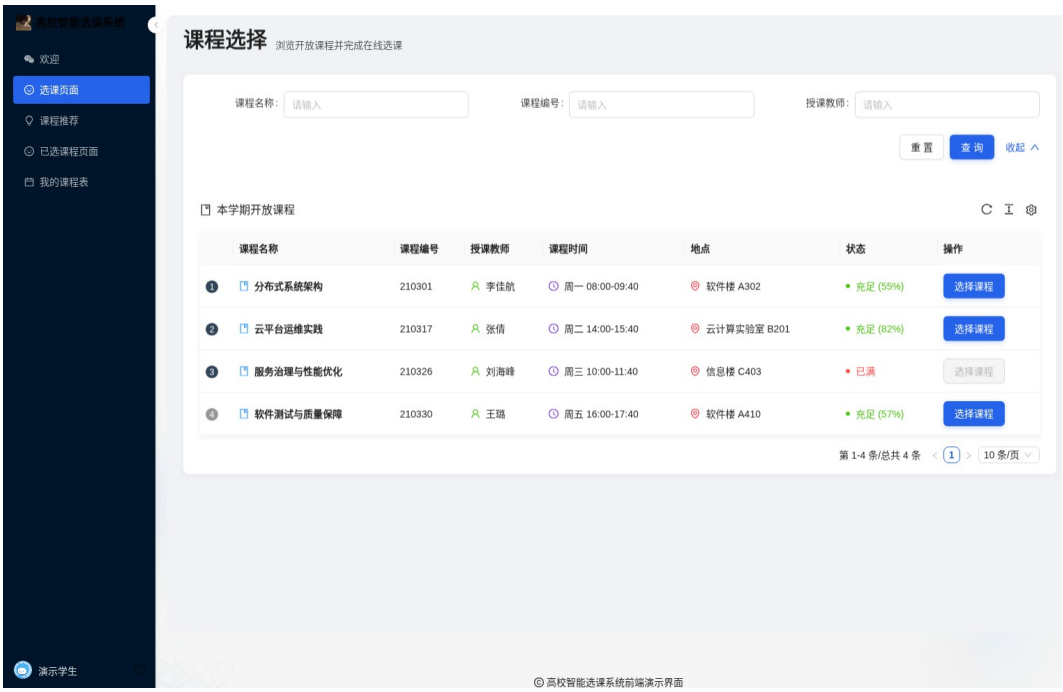


图 5.2 选课页面

图 5.3 选课操作确认弹窗

5.2.3 选课结果通知页面

学生的所有选课结果均通过站内消息中心统一发布。当前实现下，通知消息以结果提示和失败原因说明为主；在重复选课等部分场景中，系统还会追加课程名称，帮助学生及时判断后续是否需要调整选课方案。选课结果通知页面如图 5.4 所示。

图 5.4 选课结果通知页面

5.2.4 统一登录与网关转发流程

系统采用统一登录页与网关令牌校验的访问控制机制。用户首次请求受保护资源时，Gateway 检测到请求未携带 Token，会先将用户重定向到统一登录页，并在回跳地址参数中保留原始目标路径；用户提交账号密码后，主系统完成认证并生成 Token 写入 Redis，前端保存 Token 后再回跳到最初访问的业务地址；Gateway 完成 Token 校验后，才会把请求路由到具体业务服务。整个统一登录与网关转发流程如图 5.5 所示。

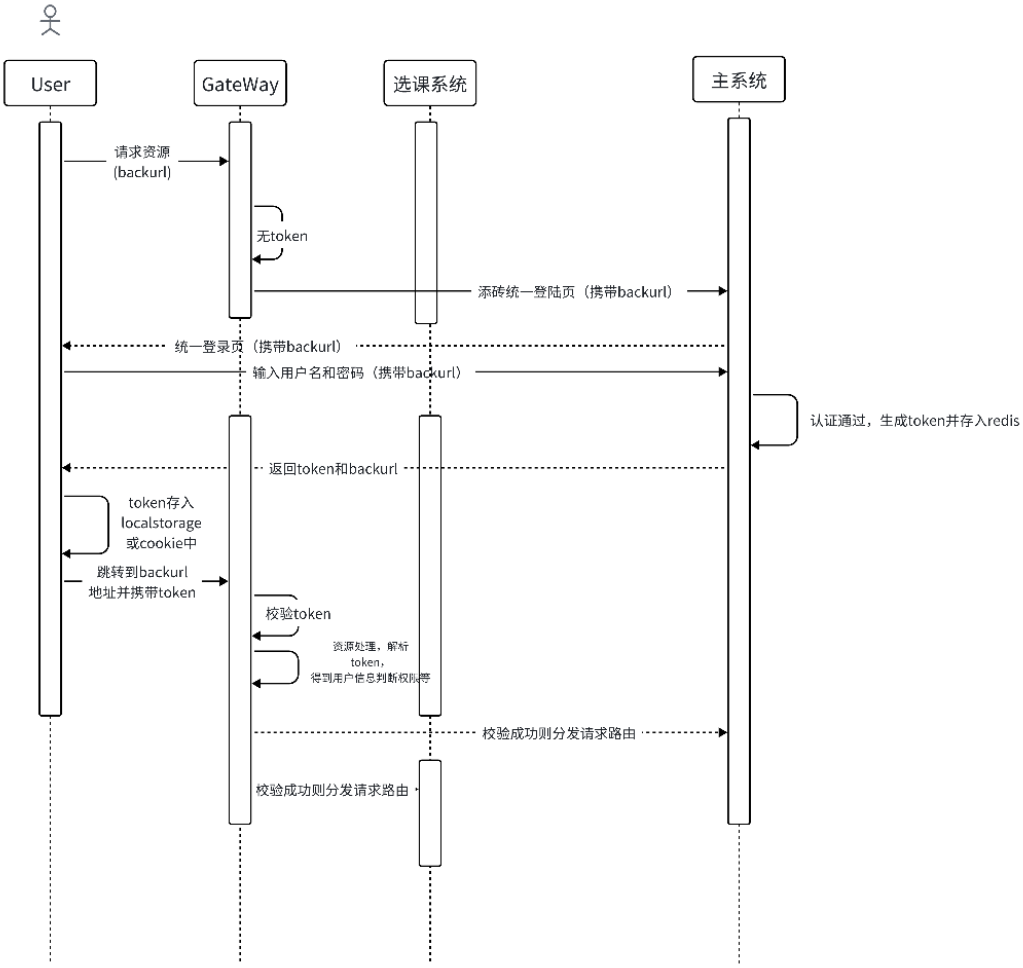


图 5.5 统一登录与网关转发流程图

5.3 课程冲突检测模块

5.3.1 冲突检测算法实现

冲突检测核心实现位于 CourseConflictServiceImpl 类的 isTimeConflict 方法。该方法通过时间区间扩展与三条件交叉判断，对课程时间重叠的主要场景进行预警。完整代码如下。

```
private static final long CONFLICT_THRESHOLD_MINUTES = 15;

@Override
public boolean isTimeConflict(Curriculum c1, Curriculum c2) {
    Date t1 = c1.getTeachingTime();
    Date t2 = c2.getTeachingTime();
    if (t1 == null || t2 == null) return false;

    long duration = TimeUnit.HOURS.toMillis(2); // 课时2小时
    Date end1 = new Date(t1.getTime() + duration);
    Date end2 = new Date(t2.getTime() + duration);
```

```
        // 对 c1 的区间两端各扩展 15 分钟缓冲
        long buf = TimeUnit.MINUTES.toMillis(CONFLICT_THRESHOLD_MINUTES);
        Date adjStart1 = new Date(t1.getTime() - buf);
        Date adjEnd1   = new Date(end1.getTime() + buf);

        // 三条件覆盖所有时间重叠场景
        return (t2.after(adjStart1)   && t2.before(adjEnd1))
            || (end2.after(adjStart1) && end2.before(adjEnd1))
            || (t2.before(adjStart1)  && end2.after(adjEnd1));
    }
```

三个判断条件的物理含义：条件一覆盖"c2 开始时间落在 c1 扩展区间内"的场景；条件二覆盖"c2 结束时间落在 c1 扩展区间内"的场景；条件三覆盖"c2 时间完全包含 c1 扩展区间"的极端场景。三者组合能够覆盖课程时间明显重叠的主要情况。15 分钟缓冲阈值贴合高校实际情况，两节课之间若间隔不足 15 分钟，学生往往难以在不同教学楼间完成转场，因此将此类情况纳入冲突预警范围。

冲突原因描述根据时间差进行差异化生成：时间差为 0 时提示"两门课在同一时间开始"；时间差在 15 分钟以内时提示"两门课时间相隔仅 N 分钟，可能来不及转换教室"；超出缓冲但存在区间重叠时提示"课程时间重叠"或"时间接近，可能无法及时到达"。这种解释性提示有助于学生理解冲突性质，做出更稳妥的选课决策。

5.4 系统管理模块的实现

5.4.1 学生管理页面

学生管理页面用于管理员维护学生档案，界面采用 Ant Design ProTable 展示列表，支持按学号、姓名、专业等多条件筛选，操作区提供新增、编辑、删除入口。新增学生时，系统会自动在 user 表中创建对应账户并关联 student 角色。删除操作采用逻辑删除（isDelete 置 1），不直接物理移除数据，便于后续历史查询与问题排查。学生管理页面如图 5.6 所示。

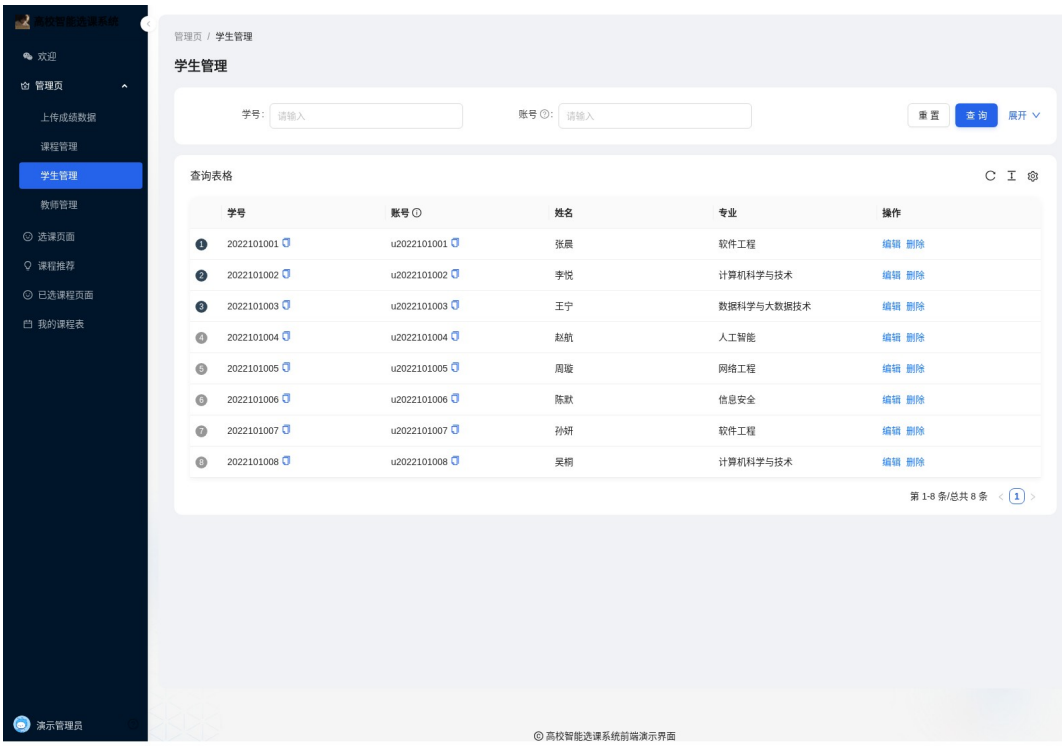


图 5.6 学生管理页面

5.4.2 课程管理页面

课程管理实现了二级审核流程：教师填写课程信息表单（课程名称、学分、授课时间、地点、专业方向、名额上限等字段）提交后，系统将 isCheck 置为 0（待审核），课程在审核通过（isCheck=1）前不对学生可见；管理员在课程管理页面执行通过、驳回（需填写驳回原因）操作；被驳回课程经教师修改后可重新提交审核。课程管理页面如图 5.7 所示。

图 5.7 课程管理页面

5.4.3 教师管理页面

教师管理页面提供教师档案的增删改查功能，字段包括工号、姓名、职称等。新增教师的处理逻辑与新增学生类似，管理员提交表单后，后台同步在 user 表中创建账户并绑定 teacher 角色，教师随后即可使用该账号登录并发布课程。教师管理页面如图 5.8 所示。

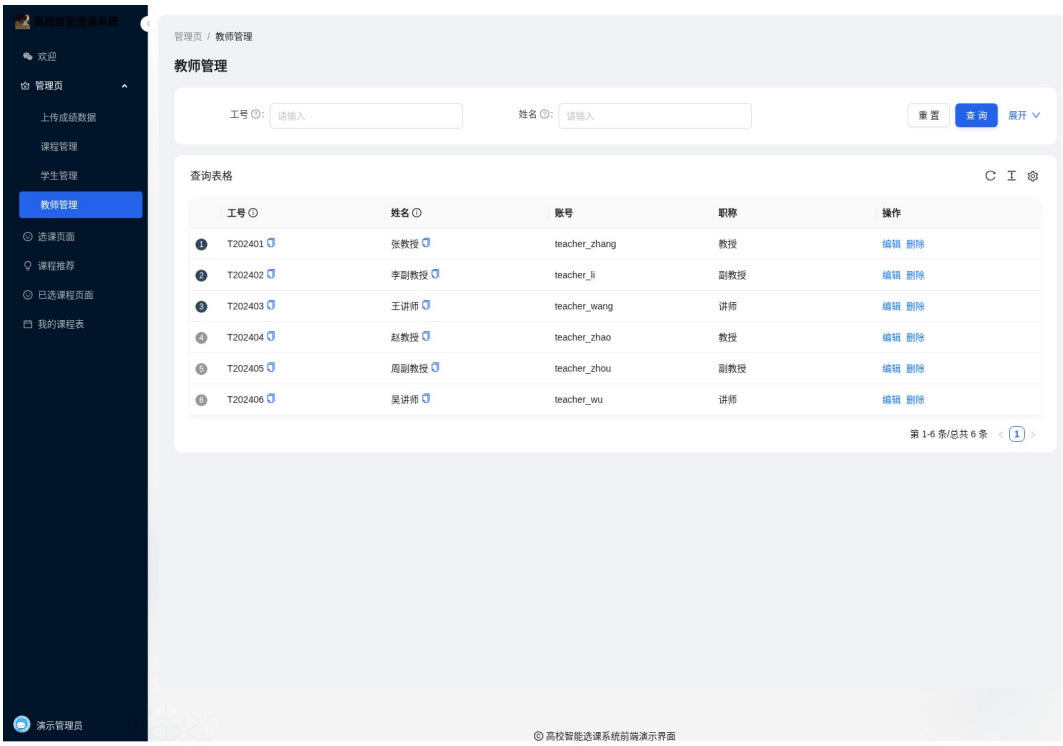


图 5.8 教师管理页面

5.5 消息通知模块的实现

消息通知模块采用 RabbitMQ 实现异步推送。选课处理完成后，消费者根据结果构建通知消息并发送至通知队列，通知消费者获取消息后写入 message 表，同时对用户的 unReadMessage 字段执行加 1 处理。前端采用定时轮询方式，每隔 30 秒请求一次未读数接口，若存在新消息，则在导航栏左上角显示红点数字。

当前实现下，成功通知通常展示"选课成功"等结果提示，失败通知则明确说明失败原因，比如"课程已满"或者"你已选过该课程"。在重复选课这类场景中，系统还会附带课程名称，帮助学生快速定位是哪门课触发了提示，如图 5.9 和图 5.10 所示。

图 5.9 选课成功通知示例

图 5.10 选课失败通知示例

5.6 系统安全模块的实现

5.6.1 Sa-Token 登录与 Token 管理

安全模块的鉴权工作主要由 Sa-Token 完成。用户登录时，前端将账号密码 POST 至/api/user/login，后端首先进行非空校验，随后调用 userService.userLogin()对输入密码执行加盐 MD5 处理，并与数据库中的密文进行比对；比对通过后调用 StpUtil.login(userId)创建会话，会话信息结合 Redis 进行存储，并在返回前端时一并携带用户信息与角色标识，以支持前端完成权限展示控制。登录核心代码如下。

```
@PostMapping("/login")
public BaseResponse<Map<String,Object>> userLogin(
        @RequestBody UserLoginRequest req, HttpServletRequest request) {
    if (req == null)
        throw new BusinessException(ErrorCode.PARAMS_ERROR);
    String account  = req.getUserAccount();
    String password = req.getUserPassword();
    if (StringUtils.isAnyBlank(account, password))
        throw new BusinessException(ErrorCode.PARAMS_ERROR);
    User user = userService.userLogin(account, password, request);
    LoginUserVO loginUserVO = userService.getLoginUserVO(user);
    SaTokenInfo tokenInfo  = StpUtil.getTokenInfo();
    Map<String,Object> result = new HashMap<>();
    result.put("user",       loginUserVO);
    result.put("tokenName",  tokenInfo.getTokenName());
    result.put("tokenValue", tokenInfo.getTokenValue());
    result.put("role",       user.getUserRole());
    return ResultUtils.success(result);
}
```

5.6.2 网关层鉴权过滤器

Spring Cloud Gateway 通过自定义 SaReactorFilter 拦截所有入站请求，对登录注册以及当前预留的验证码路径直接放行，其余接口调用 StpUtil.checkLogin()进行 Token 有效性验证。Sa-Token 从 HTTP 请求头中提取 Token 值，到 Redis 中查询对应会话信息，若 Token 不存在或已过期则返回错误信息。鉴权过滤器同时处理 CORS 预检请求（OPTIONS 方法），通过 setHeader 注入跨域响应头后直接放行，避免浏览器跨域限制影响前端正常请求。

```
@Bean
public SaReactorFilter saReactorFilter() {
    return new SaReactorFilter()
        .addInclude("/**")
        .addExclude(
            "/favicon.ico",
            "/api/user/login",
```

```
                "/api/user/register",
                "/api/user/captcha"
            )
            .setAuth(obj -> {
                SaRouter.match("/**", StpUtil::checkLogin);
            })
            .setBeforeAuth(obj -> {
                SaHolder.getResponse()
                    .setHeader("Access-Control-Allow-Origin",
                        "http://localhost:8000")
                    .setHeader("Access-Control-Allow-Methods","*")
                    .setHeader("Access-Control-Allow-Headers",
                        "authorization, content-type")
                    .setHeader("Access-Control-Allow-Credentials",
                        "true")
                    .setHeader("Access-Control-Max-Age","3600");
                SaRouter.match(SaHttpMethod.OPTIONS).free(r->{}).back();
            });
    }
```

当前配置中 Token 默认有效期为 30 天，active-timeout 设置为-1，不启用无操作冻结机制，整体更偏向于保证开发与演示阶段的登录连续性。Token 结合 Redis 存储后，任意一个服务节点都可以独立完成验证，不依赖某台固定机器。通过网关鉴权后，用户即可访问受保护的课程表页面，如图 5.11 所示。

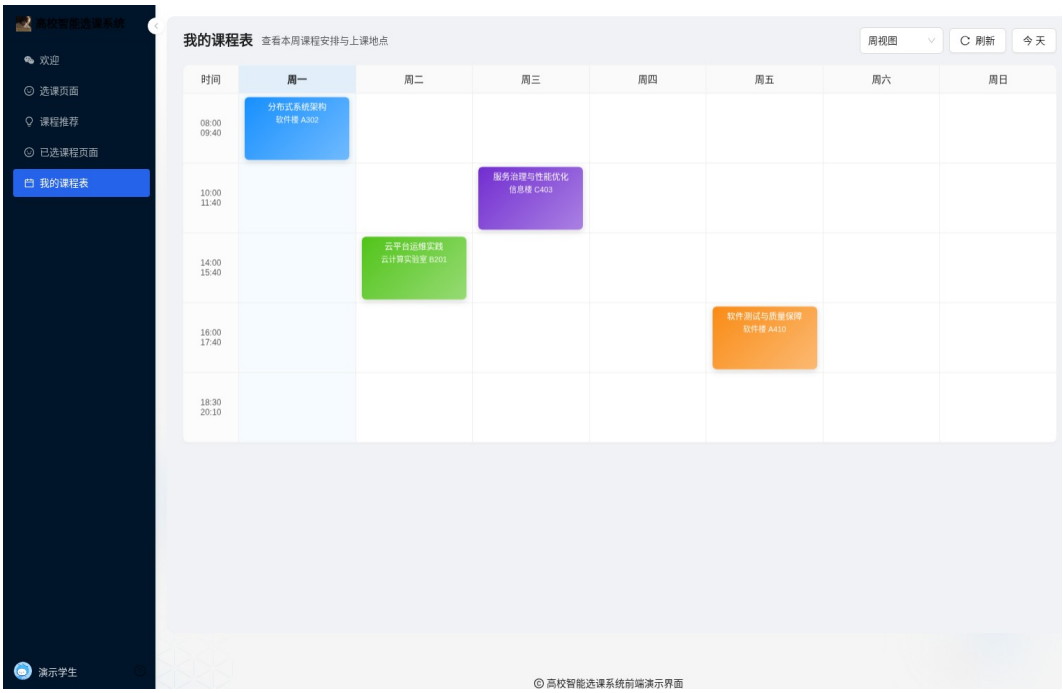


图 5.11 我的课程表页面

5.7 本章小结

本章以六大功能模块为核心，通过关键代码解析与系统界面截图相结合的方式，系统性呈现了智能推荐、高并发选课防护、时间冲突检测、系统管理、消息通知和安全鉴权模块的实现细节。智能推荐模块通过专业匹配评分实现了轻量级的个性化推荐；高并发选课模块通过 Redisson 锁、处理日志与 RabbitMQ 队列协同控制并发压力；冲突检测模块基于时间窗口扩展算法对主要重叠场景进行预警；安全模块实现了统一登录态校验与会话管理。各模块协同工作共同支撑了系统在高并发、高可靠场景下的运行。

第 6 章 系统测试

6.1 测试设计

系统测试是验证软件功能正确性与非功能性指标合规性的关键环节。本文采用多层次测试策略：通过手动黑盒功能测试验证各模块业务逻辑的正确性；通过 JMeter 性能压力测试评估系统在高并发场景下的响应能力与稳定性，为优化提供量化数据[11]。测试覆盖系统安全（登录鉴权）、智能推荐、选课流程、冲突检测、系统管理、消息通知六大功能模块，以及高并发选课场景下的性能指标验证。

6.2 主要功能测试

6.2.1 系统安全功能测试

系统安全功能测试重点验证用户认证机制、登录态校验与受保护接口访问控制的有效性，测试用例如表 6.1 所示。

表 6.1 系统安全模块测试用例表

编号	测试内容	输入	预期输出	结果
A1	正常用户登录	正确账号和密码	返回 Token 及用户信息，跳转首页	通过
A2	错误密码登录	正确账号+错误密码	提示账号或密码错误	通过
A3	未登录访问受保护接口	无 Token 请求	返回 401 未授权错误	通过
A4	Token 过期访问	过期 Token 请求	返回 401，提示重新登录	通过
A5	注销后访问受保护接口	已退出登录的 Token 请求	返回 401，需要重新登录	通过
A6	用户注册	新账号+密码	注册成功，跳转登录页	通过
A7	查看个人信息	已登录用户	正确展示用户个人信息	通过

6.2.2 智能推荐功能测试

智能推荐功能测试验证推荐算法输出的准确性与推荐理由展示的合理性，测试用例如表 6.2 所示。

表 6.2 智能推荐模块测试用例表

编号	测试内容	输入	预期输出	结果
B1	专业匹配推荐优先展示	软件工程专业学生	专业相关课程排名靠前（分数 90）	通过

B2	推荐数量限制	limit=5 参数	返回恰好 5 门推荐课程	通过
B3	推荐理由展示	查看推荐列表	每门课程正确显示推荐理由文字	通过
B4	无匹配课程场景	小众专业学生	返回通用选修课程列表（分数 50）	通过
B5	空学生 ID	studentId 不存在	返回空列表而非异常	通过

6.2.3 选课模块功能测试

选课模块测试重点验证高并发防护机制的正确性与边界条件处理，测试用例如表 6.3 所示。

表 6.3 选课模块测试用例表

编号	测试内容	输入	预期输出	结果
C1	正常选课流程	有效 studentId 和 curriculumId	返回处理中，结果通过消息通知	通过
C2	内部请求日志重复保护	相同 requestId 重复提交 3 次	第 2、3 次返回已处理结果，不重复选课	通过
C3	课程已满防超卖	stock=0 的课程	返回"选课失败,该课程人数已满"	通过
C4	重复选课拦截	已选过的 curriculumId	返回"选课失败，你已经选过该课程"	通过
C5	退课功能	有效 studentId 和 courseId	退课成功，stock 递增 1 恢复	通过
C6	无效课程 ID	不存在的 curriculumId	返回合理的错误提示	通过

6.2.4 冲突检测功能测试

冲突检测测试验证时间窗口算法在常见时间边界条件下的判断表现，测试用例如表 6.4 所示。

表 6.4 冲突检测模块测试用例表

编号	测试内容	输入	预期输出	结果
D1	无冲突场景	时间间隔>2 小时 30 分的两门课	检测结果：无冲突	通过
D2	完全时间重叠	同一时间开始的两门课	检测到冲突，提示同一时间开始	通过
D3	缓冲区内冲突	时间间隔 10 分钟的两门课	检测到冲突，提示相隔 10 分钟	通过
D4	缓冲边界值	时间间隔 16 分钟	无冲突（边界值	通过

		（超出阈值）	测试通过）	
D5	部分时间重叠	c2 开始时间落在 c1 结束前	检测到冲突，提示时间重叠	通过
D6	c2 完全包含 c1	c2 时间段涵盖 c1 时间段	检测到冲突	通过

6.2.5 系统管理模块功能测试

系统管理模块测试验证学生、课程、教师三类数据的 CRUD 操作及审核流程的正确性，测试用例如表 6.5 所示。

表 6.5 系统管理模块测试用例表

编号	测试内容	输入	预期输出	结果
E1	新增学生	有效学生信息	新增成功，自动创建关联用户账号	通过
E2	删除学生（逻辑删除）	有效学生 ID	isDelete 置 1，数据不物理删除	通过
E3	课程审核通过	待审核课程 ID	isCheck 置 1，课程对学生可见	通过
E4	课程审核驳回	待审核课程 ID+驳回原因	isCheck 置 2，教师收到驳回通知	通过
E5	课程强制扩容	课程 ID+新名额数	stock 更新为新值	通过
E6	查询学生列表	分页+关键词筛选	返回符合条件的分页数据	通过

6.2.6 消息通知模块功能测试

消息通知模块测试验证通知投递的及时性与未读状态管理的正确性，测试用例如表 6.6 所示。

表 6.6 消息通知模块测试用例表

编号	测试内容	输入	预期输出	结果
F1	选课成功通知	成功选课后	消息中心收到成功结果提示	通过
F2	选课失败通知	课程已满时选课	消息中心收到含失败原因的失败通知	通过
F3	未读消息计数更新	有新通知时	导航栏红点显示正确未读数量	通过
F4	查看消息后已读	点击消息详情	unReadMessage 计数相应减少	通过

6.3 性能压力测试

性能测试在笔者本地机器上进行（AMD R7 4800U，16GB 内存），用 JMeter 5.6 模拟并发选课请求[12]，重点测的是选课接口（POST /api/enrollments/select）。测试方法是把思考时间设为 0，每个虚拟用户循环 5 次，这样能产生最大的瞬时并发压力。测试指标关注平均响应时间、吞吐量和错误率，超卖次数单独统计。并发性能测试数据如表 6.7 所示。

表 6.7 并发性能压力测试数据表

并发用户数	平均响应时间(ms)	吞吐量(TPS)	错误率	库存超卖次数
100	180	560	0%	0
300	290	980	0%	0
500	420	1050	0.1%	0
800	610	1180	0.8%	0
1000	790	1218	1.8%	0
1500	1350	1100	4.5%	0

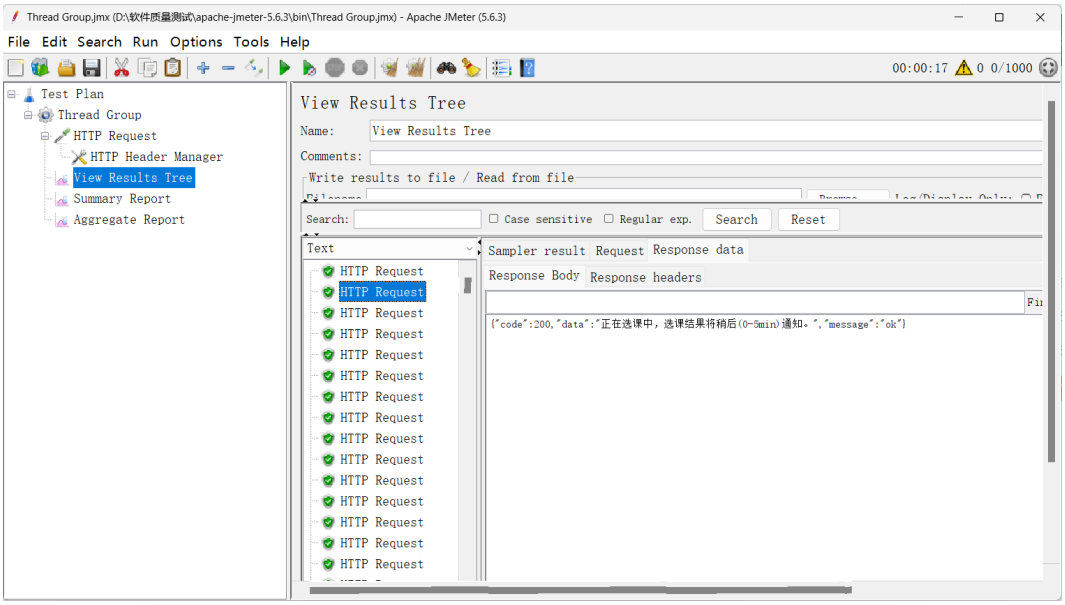


图 6.1 JMeter 并发压力测试结果图

测试结果表明，在 100 至 500 并发区间内，系统平均响应时间保持在 500ms 以内，吞吐量随并发规模增加而基本呈线性增长，整体运行较为稳定。当并发提升至 1000 时，平均响应时间为 790ms，吞吐量达到 1218 TPS，错误率为 1.8%，能够满足高校选课场景的基本需求。当并发进一步提升至 1500 时，响应时间超过 1 秒，错误率上升至 4.5%，说明当前单机部署模式的性能上限大致位于 1200 至 1500 并发区间。所有测试场景中库存超卖次数均为 0，说明 Redisson 分布式锁对临界区具有较好的保护效果。

6.4 本章小结

本章从功能测试与性能测试两个方面对系统进行了验证。34 个功能测试用例全部通过，表明各模块的业务逻辑与边界处理符合预期。性能测试结果显示，在 1000 并发条件下系统平均响应时间为 790ms、吞吐量为 1218 TPS、错误率为 1.8%，且测试过程中未出现库存超卖现象。综合来看，系统在功能正确性与高并发稳定性方面均达到了既定目标。

第 7 章 总结与展望

7.1 总结

本文主要围绕传统高校选课系统的三个突出问题展开：其一，选课高峰期系统稳定性不足；其二，缺乏个性化推荐导致学生选课效率较低；其三，课程时间冲突往往在选课完成后才被发现。针对上述问题，本文基于 Spring Boot 2.6.13 和 Spring Cloud Alibaba 搭建微服务架构，使用 Nacos 实现服务注册与配置管理，使用 Spring Cloud Gateway 作为统一路由与鉴权入口，使用 Sa-Token 完成登录态校验，使用 Redis 与 Redisson 保障并发安全，使用 RabbitMQ 承担异步削峰和通知投递任务，前端采用 React 与 Ant Design Pro 构建界面，数据层采用 MyBatis-Plus 降低重复开发成本。

在技术实现层面，系统形成了三项较为明确的工程成果：其一，基于专业匹配的轻量级智能推荐算法，无需复杂机器学习模型即可为学生提供可解释的个性化课程推荐，算法复杂度为 O(n log n)；其二，基于时间窗口扩展的课程冲突检测算法，引入 15 分钟转场缓冲阈值，更贴近高校实际场景，能够对主要时间重叠情况给出预警；其三，Redisson 分布式锁、处理日志与 RabbitMQ 队列组合形成了高并发选课处理链路，在 1000 并发用户的压力测试中实现零超卖、1218 TPS 吞吐量。

34 个功能测试用例全部通过，JMeter 压力测试结果亦处于预期范围内，说明系统在功能完整性与性能稳定性方面均达到了设计目标。该系统对高校教务信息化建设具有一定的参考价值，尤其适用于中小规模高校的选课系统改造场景，其技术方案成熟度较高，实施成本也相对可控。

7.2 展望

尽管本系统已具备较高的功能完整性与系统稳定性，仍有多个方向值得进一步探索。

（1）推荐算法深化：当前专业匹配推荐算法较为简单，仅考虑学生专业这一维度。未来可引入协同过滤算法（基于相似学生群体历史选课行为推荐）或矩阵分解模型（如 SVD），通过分析学生的历史选课记录、成绩分布和选课时间偏好，构建更精准的多维度个性化推荐模型，进一步提升推荐准确性和学生满意度。

（2）弹性伸缩增强：当前系统基于单机 Docker Compose 部署，未引入 Kubernetes 容器编排。未来可迁移至 Kubernetes 集群，配置 Horizontal Pod Autoscaler（HPA）根据 CPU 使用率自动调整服务副本数量，实现选课高峰期的弹性扩容与低谷期的资源回收，进一步降低运营成本并提升系统整体并发承载上限。

（3）移动端体验优化：随着移动互联网的普及，开发 React Native 或 Flutter 移动端应用，支持学生随时随地查看推荐课程、接收选课通知、查看课程表，将

大幅提升用户体验与系统可用性，满足移动学习时代的新需求。

（4）智能预测与主动扩容：引入时间序列预测模型（如 LSTM 或 Prophet），基于历史选课数据预测各时段并发请求量，在预测高峰到来前提前完成资源扩容预热，将被动弹性伸缩转变为主动资源预置，从根本上消除高峰初期的响应延迟问题。

（5）数据安全增强：对选课记录引入数字签名或区块链存证机制，确保选课数据不可篡改，为成绩认定、学分互认等场景提供权威数据依据，提升教务数据的合规性与可信度。

总体而言，本系统为高校选课管理提供了具备实践价值的现代化智能解决方案。随着持续的技术迭代与功能演进，有望发展成为更智能、更高效、更安全的综合教务管理平台，为高等教育信息化进程贡献技术力量。

参考文献

[1] 曾佳. 基于 Spring Cloud 的微服务架构设计[J]. 电子技术, 2023, 52(01): 54-55.

[2] 李俊俊, 董建刚, 李坤. 基于 Kubernetes 的集群节能策略研究[J]. 计算机工程, 2024, 50(9): 82-91.

[3] 林航. Docker 容器技术在微服务架构中的应用[J]. 网络安全和信息化, 2024.

[4] 贺敬伟, 程伟华, 张世杰. 基于 Kubernetes 调度算法的动态负载均衡方法研究[J/OL]. 自动化技术与应用.

[5] 王凯旋. 基于三支决策的容器伸缩机制研究[D]. 哈尔滨师范大学, 2020.

[6] 王栋柱, 王青青, 陈华林, 等. 基于 Redis 的高性能分布式锁设计与实现[J]. 软件, 2024, 45(06): 4-6.

[7] 唐权, 周蓉, 张勇. RabbitMQ 消息中间件在 Spring Boot 教学中的应用[J]. 现代信息科技, 2020, 4(18): 125-127.

[8] 张旭东, 蒋厚明, 王俊, 等. 基于中间件的分布式系统幂等性问题研究[J]. 现代计算机, 2022, 28(15): 31-37.

[9] 卢万有. 基于 JWT 的 RBAC 在前后端分离项目中的设计与实现[J]. 电脑编程技巧与维护, 2025(01): 46-48.

[10] 王菊雅. 基于 Ant Design Pro 的物流系统前端开发与用户体验优化研究[J]. 软件, 2024(04): 8-10.

[11] 刘倩. 计算机软件测试方法的研究与分析——以集成测试为例[J]. 长江信息通信, 2023(07): 115-118.

[12] 陈建海, 陈淼, 浦云明. 基于微服务架构 B/S 系统的性能分析[J]. 计算机系统应用, 2020, 29(2): 233-237.

[13] 刘世煜. 分布式微服务架构的设计与应用[D]. 南京大学, 2021.

[14] 温立辉. 关系数据库设计原理与分析[J]. 无线互联科技, 2018.

[15] 郑松奕, 陈国良, 蒋正亮, 张裕祥. 基于 Nginx 的动态权重负载均衡技术研究[J]. 现代信息科技, 2024, 8(20): 67-71.

致谢

光阴荏苒，大学四年的求学时光如白驹过隙，转眼间毕业的钟声已悄然敲响。在这篇论文即将完成之际，我心中涌动着难以言表的感激与不舍，谨以此文向所有在我成长道路上给予帮助与支持的人致以最诚挚的谢意。

首先要向我的导师李佳航老师致以最深的敬意与感谢。从选题伊始到最终定稿，李老师始终以严谨的治学态度和深厚的专业素养给予我悉心指导。每一次组会讨论，李老师都能精准指出研究思路中的盲点，引导我从更宏观的视角审视问题；每一轮论文修改，李老师逐字推敲的认真态度让我深切感受到学术的严谨之美。正是这种言传身教，让我不仅积累了专业知识，更培养了独立思考和系统分析问题的能力，这将是我终身受益的精神财富。同时，衷心感谢孙媛老师在系统架构设计阶段给予的专业意见与悉心指点，两位老师的鼓励与支持是我顺利完成本研究的重要动力。

还要感谢学校提供的学习条件，图书馆的文献资源在文献综述阶段帮了不少忙，实验室的设备也基本满足了开发和测试的需要。大一到大三的各门专业课程是整个毕业设计的基础，数据结构让我理解了时间区间算法，分布式系统课程让我对微服务架构不再陌生，数据库课程的索引知识在性能优化阶段直接用上了，这些积累不是到了毕业设计时才发现有用，而是真的一点一点搭起来的。

感谢同窗好友们在这四年里的陪伴与互助。图书馆里共同奋战的备考深夜，实验室中攻克技术难题的下午，课后讨论学术问题时的思维碰撞——这些珍贵的记忆已成为我大学生涯中最温暖的底色。感谢每一位在我遇到困难时伸出援手的同学，你们的热心帮助让我深刻体会到团队协作的力量与真诚相处的可贵。

最深沉的感谢要献给我的父母。是你们二十余年的无私奉献与殷切期望，为我搭建起最坚实的人生后盾。忘不了每次通话时你们质朴的叮嘱，忘不了每次回家时餐桌上的热腾腾饭菜，这些看似平凡却充满温情的画面是我在异乡求学时最珍贵的精神慰藉。你们的爱与支持是我坚持走到今天的最大动力，也是我不断进取、追求卓越的最初起点。

站在毕业的门槛前，回望四年求学之路，感恩之情油然而生。所有的汗水与付出都将化为前行的力量，所有的相遇与帮助都是人生路上最珍贵的馈赠。愿这段求学岁月永远铭记于心，愿所有关心与帮助过我的人岁月安好、幸福常伴，愿我们的明天因今日的努力而更加灿烂辉煌！